

week10报告

所有相关文件可以在 [这里](#) 找到。

所有图片中的内容，包括代码、控制台输出，均可以在该文档中找到。

题目列表

1. 过滤与映射

给定员工列表，找出所有年龄大于30岁的员工，并打印他们的姓名。

示例输出: Jerry

2. 计数操作

统计年龄大于25岁的员工数量。

示例输出: 3

3. 查找操作

找到名为"John"的员工并打印其信息。

示例输出: Employee [name=John, age=27]

4. 最大值操作

找出所有员工中的最大年龄。

示例输出: 32

5. 排序操作

将员工列表按年龄升序排序，并打印排序后的结果。

示例输出:

```
Employee [name=Amit, age=22]
Employee [name=Tom, age=24]
... (其他员工)
```

6. 字符串归约

将所有员工的姓名用逗号连接成一个字符串。

示例输出: Tom,John,Jerry,Amit,Amit

7. 分组操作

将员工按姓名分组，并按格式打印结果:

```
John:
[Employee [name=John, age=27]]
Amit:
[Employee [name=Amit, age=22], Employee [name=Amit, age=24]]
... (其他分组)
```

作答

```
// 在此处编写你的Stream代码（替换以下注释）// 任务1-7的代码需依次编写在此处
System.out.println("--- Task 1: Filter Age > 30, Print Names ---");
employeesList.stream()
    .filter(employee -> employee.getAge() > 30)
    .map(Employee::getName)
```

```

        .forEach(System.out::println);
System.out.println("-----");

System.out.println("\n--- Task 2: Count Age > 25 ---");
long countAgeGreaterThan25 = employeesList.stream()
    .filter(employee -> employee.getAge() > 25)
    .count();
System.out.println(countAgeGreaterThan25);
System.out.println("-----");

System.out.println("\n--- Task 3: Find John ---");
Optional<Employee> johnOptional = employeesList.stream()
    .filter(employee -> "John".equals(employee.getName()))
    .findFirst();
johnOptional.ifPresent(System.out::println);
System.out.println("-----");

System.out.println("\n--- Task 4: Max Age ---");
OptionalInt maxAge = employeesList.stream()
    .mapToInt(Employee::getAge)
    .max();
maxAge.ifPresent(System.out::println);
System.out.println("-----");

System.out.println("\n--- Task 5: Sort By Age ---");
employeesList.stream()
    .sorted(Comparator.comparingInt(Employee::getAge))
    .forEach(System.out::println);
System.out.println("-----");

System.out.println("\n--- Task 6: Join Names with Comma ---");
String joinedNames = employeesList.stream()
    .map(Employee::getName)
    .collect(Collectors.joining(","));
System.out.println(joinedNames);
System.out.println("-----");

System.out.println("\n--- Task 7: Grouping By Name ---");
Map<String, List<Employee>> employeesGroupedByAge = employeesList.stream()
    .collect(Collectors.groupingBy(Employee::getAge));
employeesGroupedByAge.forEach((name, group) -> {
    System.out.println(name + ":");
    System.out.println(group);
});
System.out.println("-----");

```

控制台输出

```
/home/fqwqf/.jdk/openjdk-24/bin/java -javaagent:/home/fqwqf/桌面/idea-IC-
243.25659.59/lib/idea_rt.jar=34451 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -
Dsun.stderr.encoding=UTF-8 -classpath /home/fqwqf/桌
面/project/WHU_00P_2025/week10/out/production/week10 StreamOperationOfEmployees
--- Task 1: Filter Age > 30, Print Names ---
Jerry
-----

--- Task 2: Count Age > 25 ---
2
-----

--- Task 3: Find John ---
Employee [name=John, age=27]
-----

--- Task 4: Max Age ---
32
-----

--- Task 5: Sort By Age ---
Employee [name=Amit, age=22]
Employee [name=Tom, age=24]
Employee [name=Amit, age=24]
Employee [name=John, age=27]
Employee [name=Jerry, age=32]
-----

--- Task 6: Join Names with Comma ---
Tom,John,Jerry,Amit,Amit
-----

--- Task 7: Grouping By Name ---
Tom:
[Employee [name=Tom, age=24]]
Amit:
[Employee [name=Amit, age=22], Employee [name=Amit, age=24]]
John:
[Employee [name=John, age=27]]
Jerry:
[Employee [name=Jerry, age=32]]
-----
```

总结

本次作业实践了Java Stream API在处理集合数据时的常用操作。关键点包括：

- 链式调用: 将多个操作（如 filter, map, sorted）串联起来，代码简洁易读。
- 中间操作: filter, map, sorted 等，用于转换或筛选流，返回新的流。

- 终端操作: `forEach`, `count`, `max`, `findFirst`, `collect` 等, 用于触发计算并产生最终结果或副作用。
- Lambda表达式/方法引用: 大量用于定义操作逻辑, 体现了函数式编程风格。
- Collectors: `Collectors.joining` 和 `Collectors.groupingBy` 展示了强大的数据汇总和分组能力。

总的来说, Stream API提供了一种更高效的方式来处理集合, 减少了传统的迭代和条件判断样板代码。